TECHNICAL HIRING CHALLENGE

Bot Research Engineer

Extensible Browser Bot Platform

Objective

Build an extensible web platform for defining, running, and monitoring browser bots. The first bot must use **Selenium**. Prioritize reliable execution, clear results, and reproducible setup on desktop and server environments.

Required Run Scenario

The initial Selenium bot must visit **demo.arcaptcha.co**, click the "I am not a robot" checkbox, extract **challenge_id**, and persist it in a database as part of the run history. It must then enter the configured username and password, submit the form, and check whether verification actually succeeded. Submitting the form alone must not be reported as success.

Extensible Architecture

Separate bot implementations from the platform's execution, storage, and UI layers. Provide a documented bot interface and registration mechanism so that new bots or browser automation engines can be added without rewriting the platform core. Document how to implement and register a new bot, including a minimal example. The first implementation must use Selenium; additional implementations are optional.

Platform Requirements

Web UI: List the available bots, launch a selected bot, choose **Headful** or **Headless** mode before each run, and inspect run history and results. Document how to view the browser in Headful mode on both desktop and server deployments.

Persistent history: Store the run ID, bot identity, browser mode, start and end times, challenge_id, final status, and error details. Preserve records across container restarts. Failed and blocked runs must remain visible; record challenge_id whenever it is obtained.

Docker deployment: Containerize all required services and provide a Docker Compose setup. Document prerequisites, supported operating systems and CPU architectures, and any environment-specific setup. Keep credentials and configuration outside source code.

Tests and Documentation

Test the main workflow and meaningful failures, including missing challenge_id, CAPTCHA blocking, timeouts, and unsuccessful form verification. Provide test commands and results. The README must cover setup from a clean environment, configuration, both browser modes, UI usage, data persistence, troubleshooting, and adding a new bot.

Bonus

A reproducible approach that reduces CAPTCHA blocking in this demo environment earns extra credit. Explain the approach and provide evidence of its success rate. Accurate reporting of blocked or failed attempts remains mandatory.

Submission and Evaluation

Push your code to a **GitHub repository** and submit its URL with access for reviewers. Include Docker and Compose files, a sample configuration without secrets, tests, and documentation. We will first run the project in a fresh environment using only the README. Evaluation covers extensibility, correctness, reliability, code quality, tests, and documentation. Errors that prevent setup or completion of the required workflow may disqualify the submission from a technical defense interview.